System Requirements

Contracts Information Management System (CIMS)

for the Philippine Center for Economic Development (PCED)

Client: Philippine Center for Economic Development
University of the Philippines School of Economics
Client Representative: Dr. Annette Lagman

CS 191 WFQ1 - Group LF1
Instructor: Dr. Ligaya Leah Lara-Figueroa
Software Engineering I

Group Members:
Bernadette T. Bernardino
Dominic Miguel D. Ceñido
Russell Lawrence N. Sison

October 16, 2026

1 Web App Requirements Model

1.1 Content Model

The content model describes the content objects of CIMS and how they relate. Figure 1.1a shows the data tree at a high level, and Figures 1.1b and 1.1c expand the WorkflowTemplate and Contract branches.

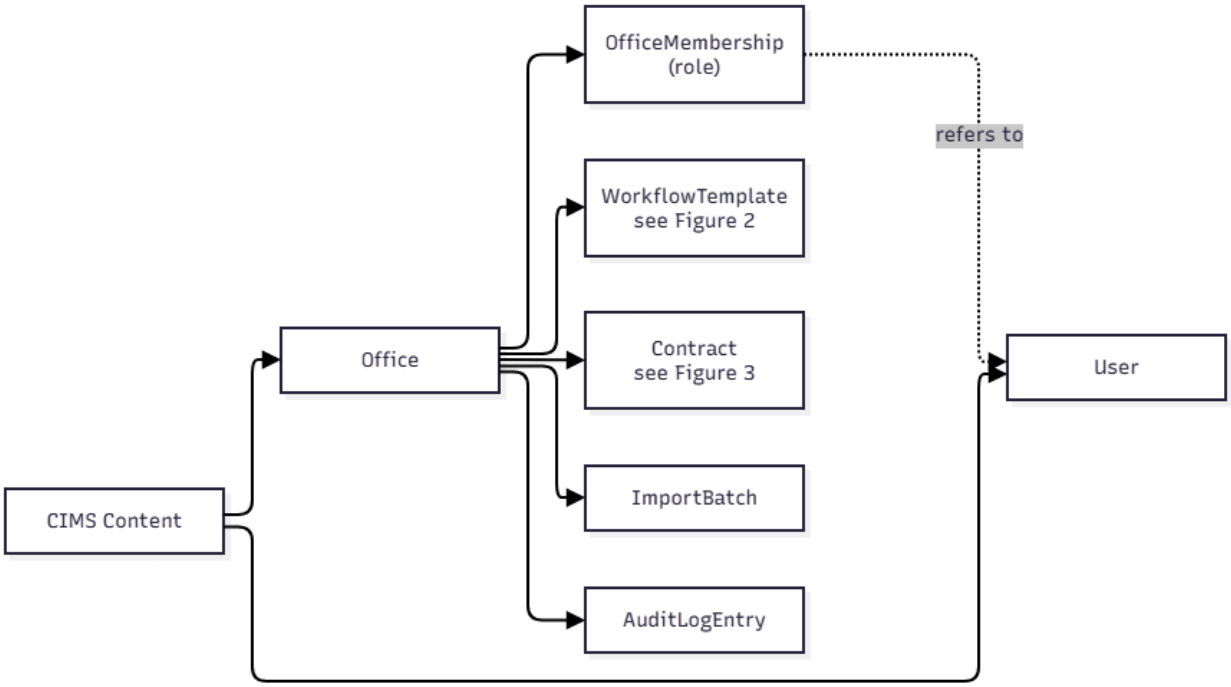


Figure 1.1a. Content data tree of CIMS (high level). Solid arrows show containment; the dotted arrow shows a reference.

Every business object belongs to an Office, which is the unit of data ownership and separation. CIMS is a general system that any office can use, and the Philippine Center for Economic Development (PCED) is the first office to use it. A User is defined once, and an OfficeMembership links a User to an Office with a role, so one person can hold different roles in different offices. Each Office owns its WorkflowTemplates, Contracts, ImportBatches, and AuditLogEntries.

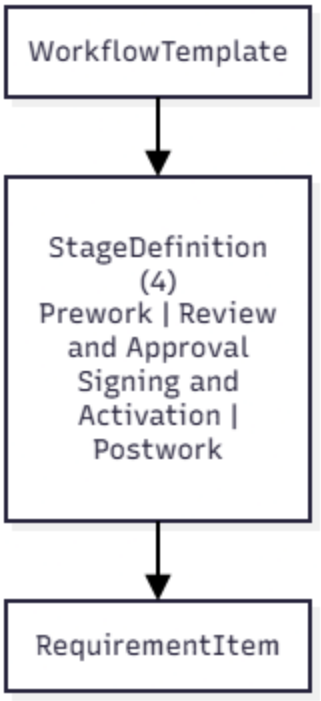


Figure 1.1b. WorkflowTemplate branch of the content data tree.

A WorkflowTemplate holds four StageDefinitions, one for each stage of the contract lifecycle: Prework, Review and Approval, Signing and Activation, and Postwork. Each StageDefinition lists its RequirementItems, which are the fields, uploads, and checks required in that stage.

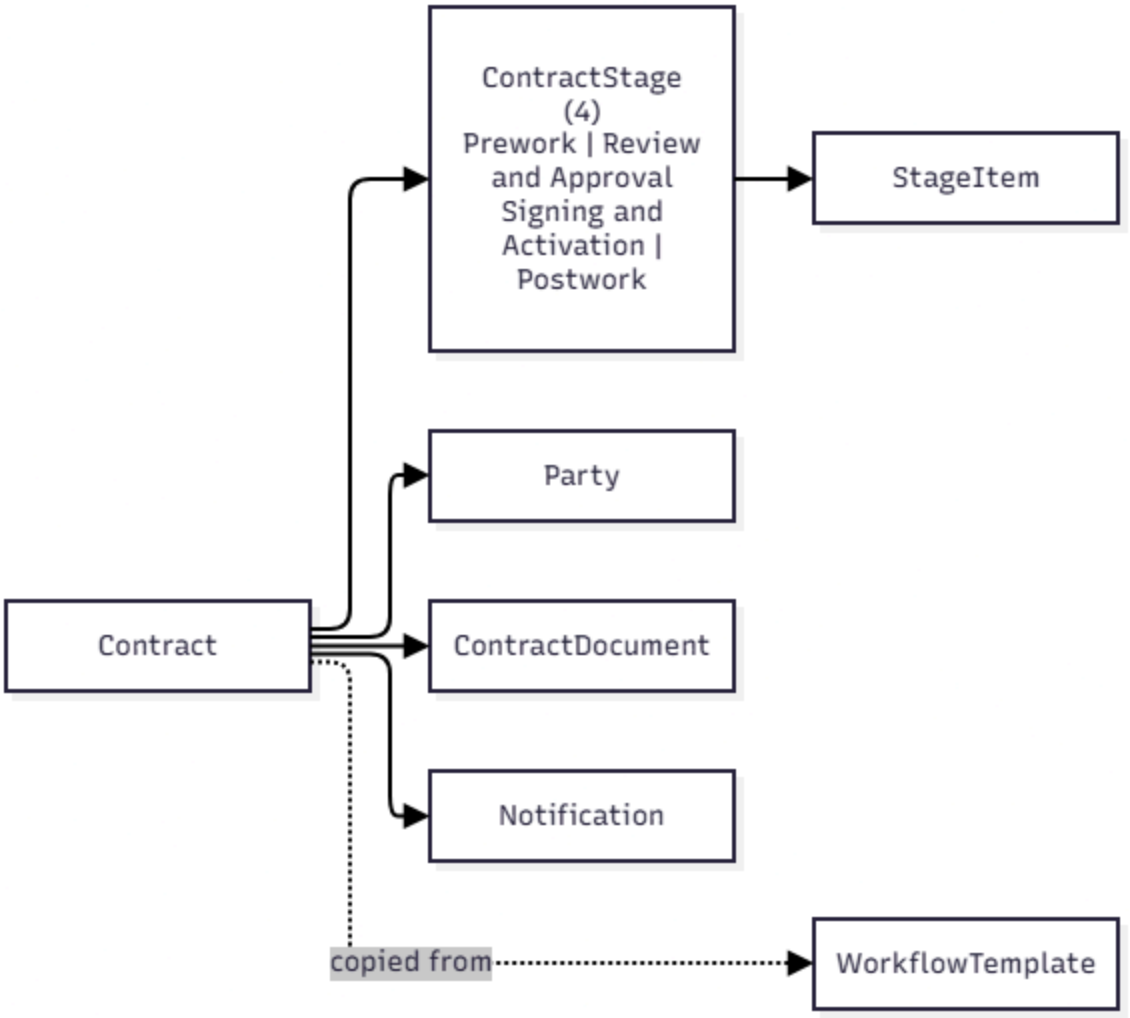


Figure 1.1c. Contract branch of the content data tree.

A Contract holds four ContractStages that match the stages of the WorkflowTemplate it was created from, and each ContractStage holds its StageItems. A Contract also holds its Parties, its uploaded ContractDocuments, and the Notifications about its deadlines. The stage requirements are copied from the WorkflowTemplate when the Contract is created, so later changes to the template do not affect existing Contracts.

1.2 Interaction Model

1.2.1 Actors and Goals

An initiating actor starts a use case to achieve a goal. A participating actor supports a use case when the system calls on it. An actor is a role: one person may play several actors, and a role is exercised within one Office.

Table 1.2.1. Actors and Goals.

Actor	Type	Goals
Contract Manager	Initiating	Create contracts from templates, record prework, signing, and postwork information, import existing records, and monitor contract status within the user's Office.
Approver	Initiating	Review contracts routed to them and approve or reject them.
Contract Viewer	Initiating	Search and view contract information in the user's Office without modifying it (read-only role).
Office Administrator	Initiating	Manage the Office's workflow templates, members and roles, reports, and audit trail.
Authentication Service	Participating	Verifies credentials and session identity through Google sign-in with Supabase Auth.
Notification Service	Participating	Delivers reminders and alerts. The delivery channels (for example email or in-app) are to be decided.
File Storage Service	Participating	Stores and returns uploaded contract documents.
Actor	Type	Goals
Contract Manager	Initiating	Create contracts from templates, record prework, signing, and postwork information, import existing records, and monitor contract status within the user's Office.

CIMS is a general contract management system that any office can use. PCED, through its Special Projects Office, is the current client and the first office to use it; the Special Projects Office is the system owner and the Office Administrator group for the PCED Office. Administrative personnel and project coordinators are expected to hold the Contract Manager, Approver, or Contract Viewer roles. There is no central administrator: each Office is managed by its own Office Administrator. How a new Office is set up in the system is to be decided.

1.2.2 Use Cases

The use cases of CIMS are described in the table below. Figure 1.2.2a groups them into four sets, and Figures 1.2.2b to 1.2.2e show the use cases of each set. Log In is used by every initiating actor before any other use case.

Table 1.2.2. Use Cases

Use case	**Initiating actor(s)**	**Brief description**
Log In	All initiating actors	The user signs in and the system verifies the account, loads the user's Office memberships, and offers only the modules the Role permits.
Manage Workflow Template	Office Administrator	The user creates, edits, saves, and retires a Workflow Template for the Office, defining the requirements of the Prework, Review and Approval, Signing and Activation, and Postwork stages.
Apply Workflow Template	Contract Manager	The user selects a Workflow Template of the Office and attaches its stage requirements to a new Contract.
Create Contract	Contract Manager	The user creates a Contract record in the Office by applying a Workflow Template.
Update Prework	Contract Manager	The user records parties, value, duration, tags, notes, and uploaded documents required by the prework stage.
Review and Approve Contract	Approver	The system routes the Contract along its review path, and the Approver records approve or reject with a checklist and notes.
Record Signing and Activation	Contract Manager	The user records signatories, signing type, notarization, and the activation date.
Track Postwork	Contract Manager	The user records and monitors monitoring requirements, milestones, deliverables, and the completion deadline.
Search and Track Contracts	Contract Manager, Approver, Contract Viewer	The user searches, filters, sorts, and opens Contracts and sees their lifecycle status, within the Office(s) the user belongs to.
Import Contracts from Spreadsheet	Contract Manager	The user uploads a spreadsheet or CSV file; the system validates it and converts the rows to structured Contract records after confirmation.
Send Deadline Notifications	Notification Service (system-timed)	The system schedules and sends reminders and alerts for deadlines and missing requirements to the responsible users.
Manage Office Users and Roles	Office Administrator	The user adds or removes OfficeMemberships and assigns roles within the Office.
Generate Office	Office Administrator	The user produces a report of the Office's

Report		Contracts by status, stage, and deadline.
View Audit Trail	Office Administrator	The user reviews the logged actions of the Office.

In the use case diagrams, light blue rectangles are initiating actors, grey rectangles are participating actors, and rounded shapes are use cases. A dotted arrow labeled extend or include is a UML «extend» or «include» relationship.

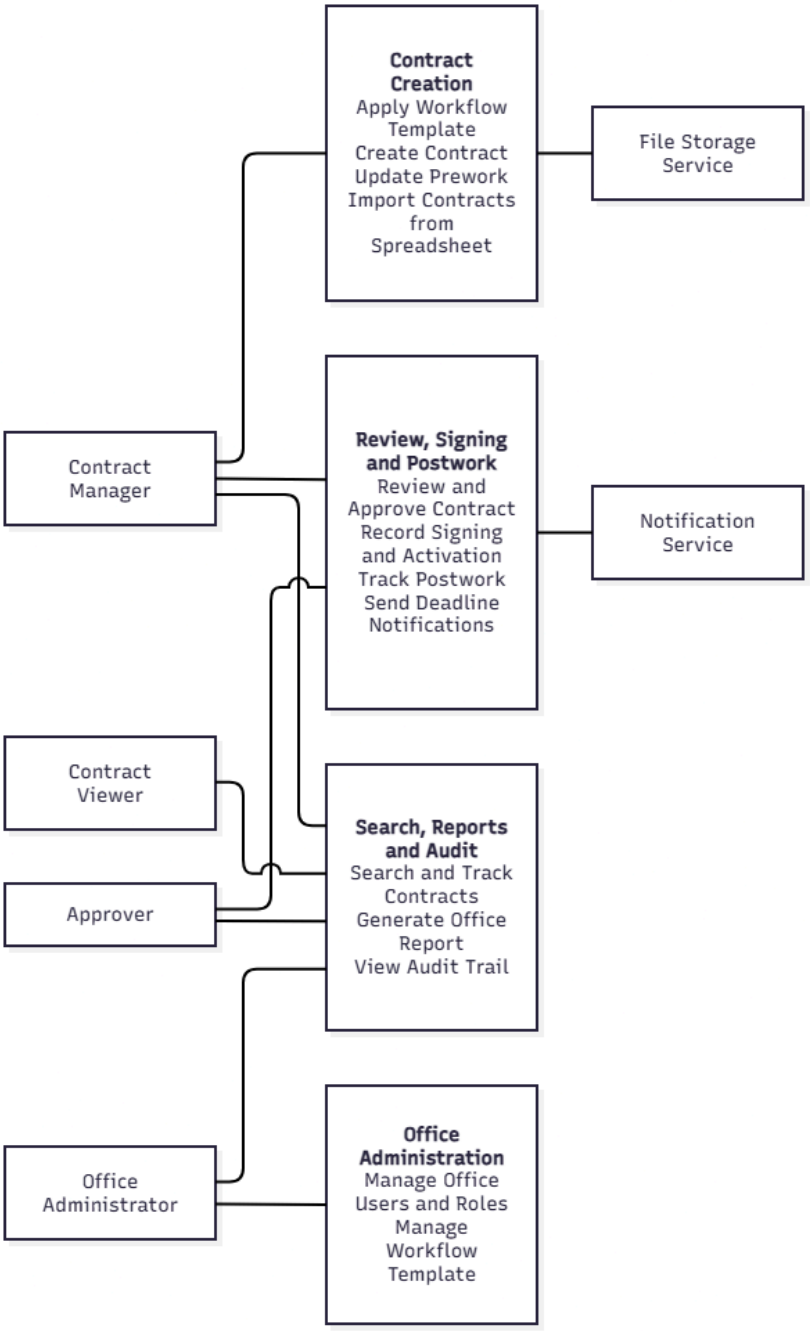


Figure 1.2.2a. Use case diagram of CIMS (high level). Each orange box is a set of use cases that is expanded in Figures 1.2.2b to 1.2.2e.

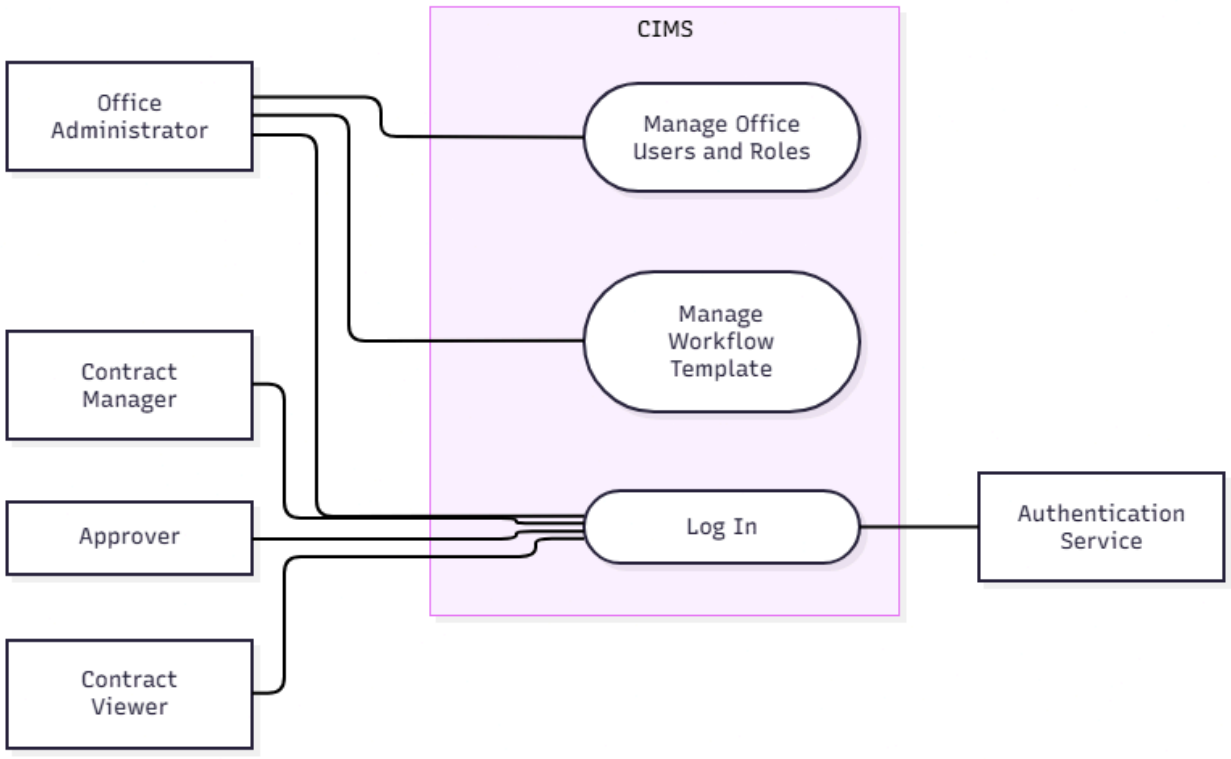


Figure 1.2.2b. Use cases for access and office administration.

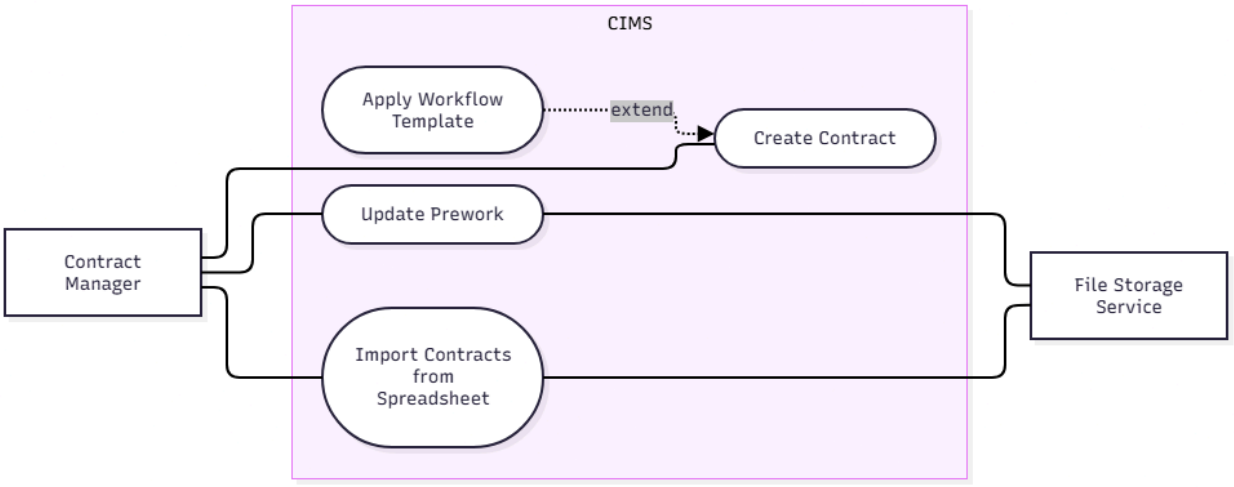


Figure 1.2.2c. Use cases for contract creation. Apply Workflow Template extends Create Contract.

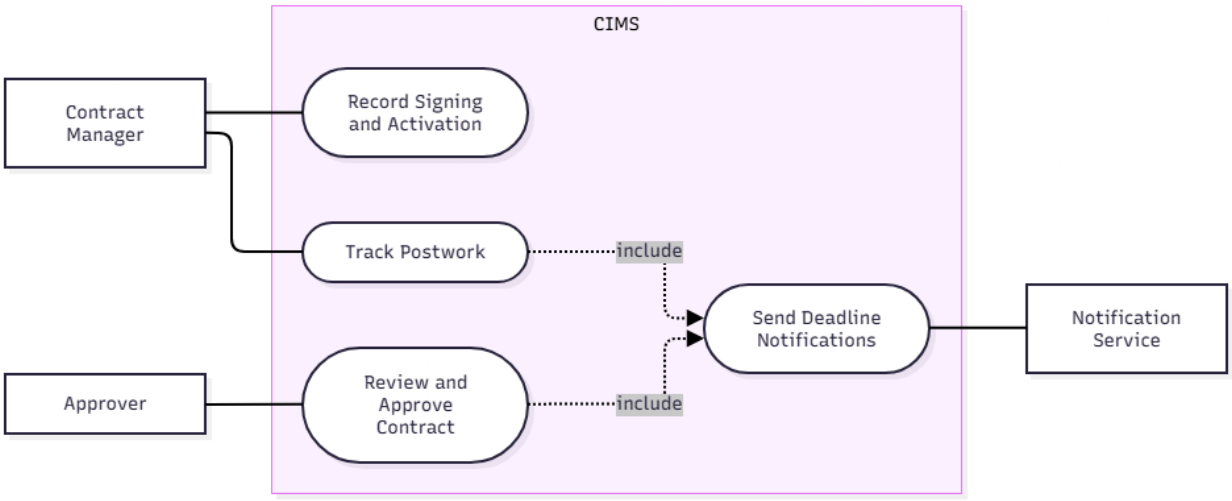


Figure 1.2.2d. Use cases for review, signing, and postwork. Review and Approve Contract and Track Postwork include Send Deadline Notifications.

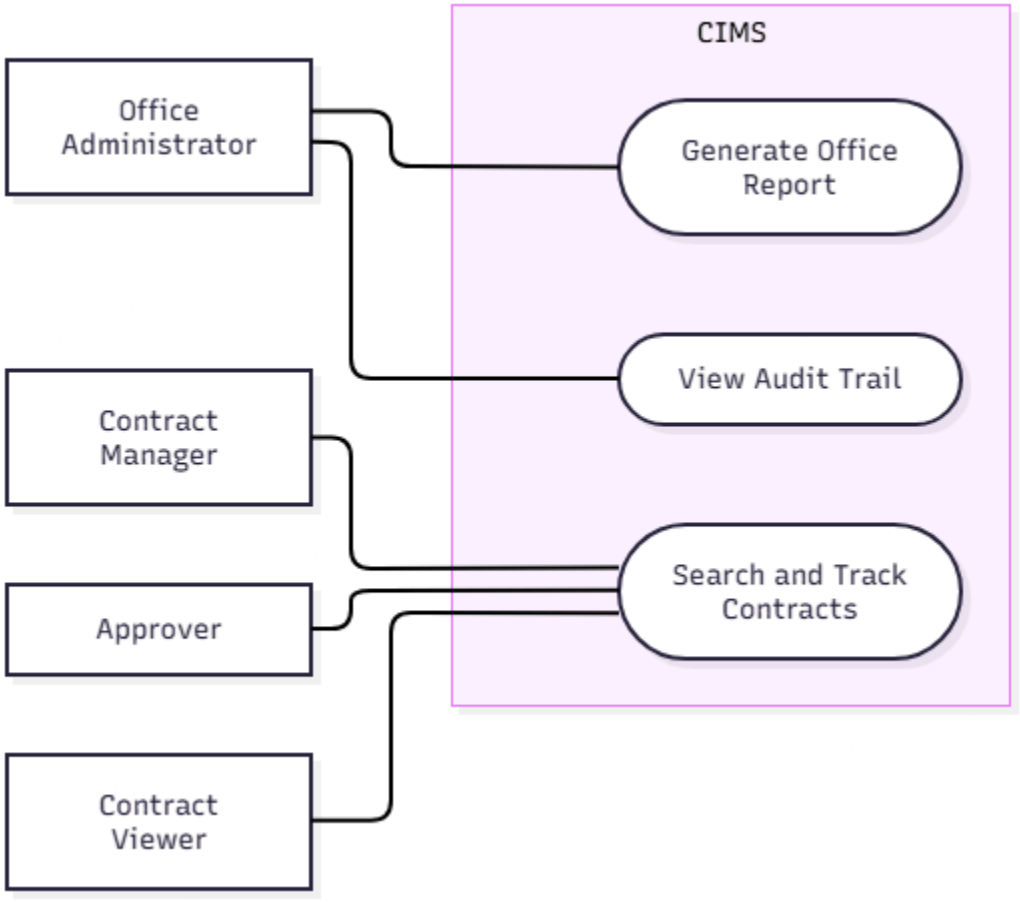


Figure 1.2.2e. Use cases for search, reports, and audit.

1.2.3 System Sequence Diagrams

Three use cases were selected because they are central to the contract lifecycle: Create Contract (with Apply Workflow Template), Review and Approve Contract, and Import Contracts from Spreadsheet. Each is shown as a main flow, followed by a second diagram for its alternative flows and checks. CIMS is shown as a single black box.

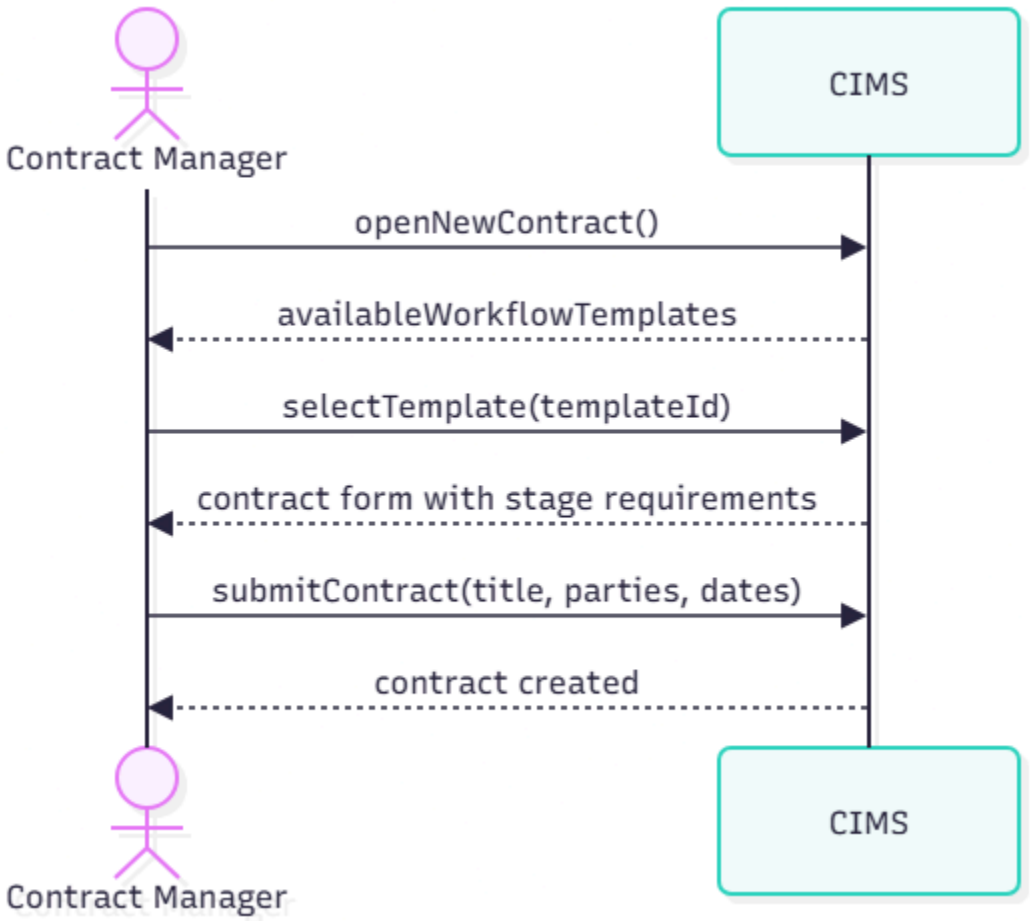


Figure 1.2.3a. System sequence diagram for Create Contract and Apply Workflow Template (main flow).

The Contract Manager can only choose from the Workflow Templates owned by the active Office, and the Contract is saved under the active Office.

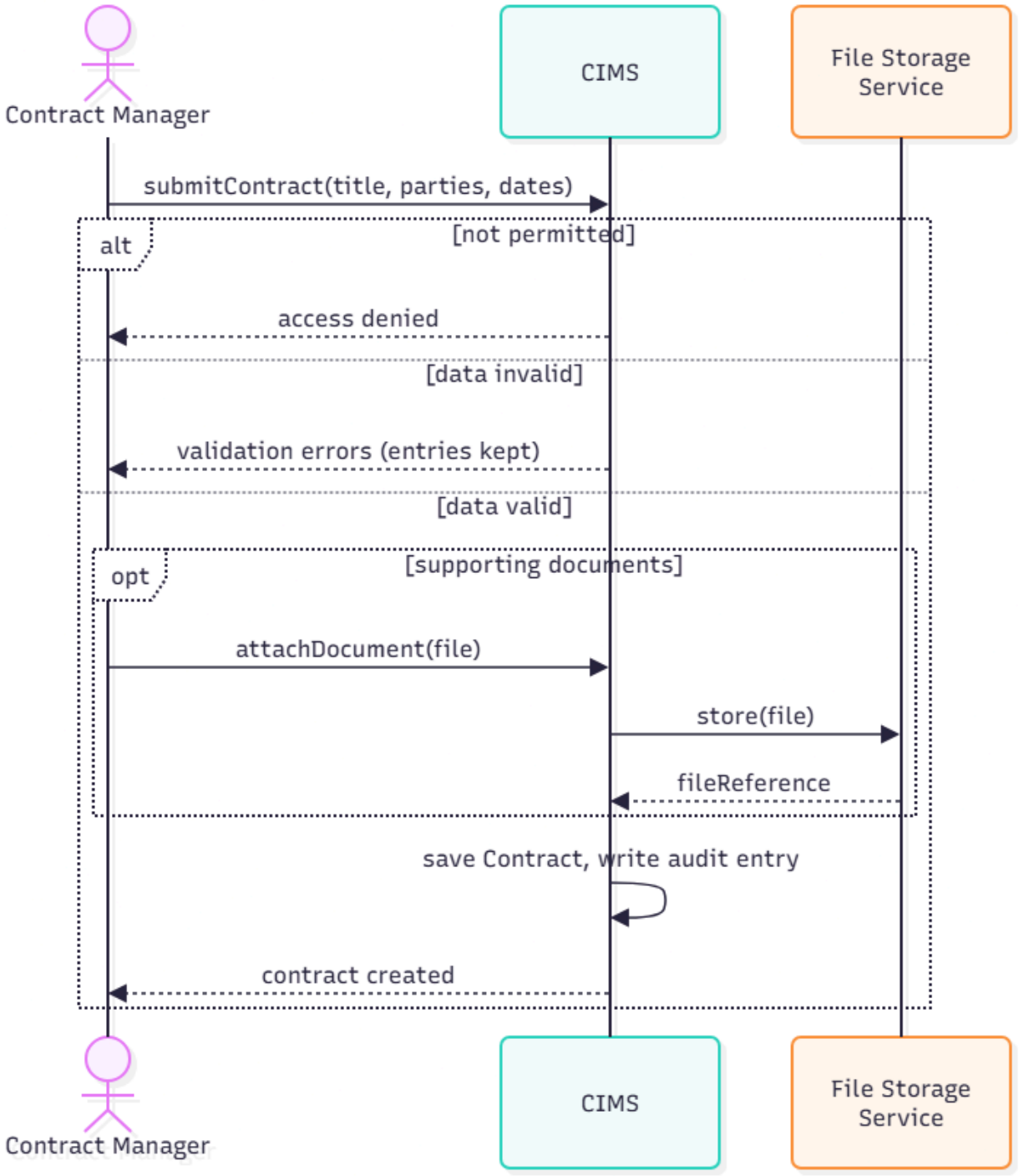


Figure 1.2.3b. Create Contract: permission check, validation, and document upload.

The permission check uses the role that the user holds in the active Office. Form entries are kept when validation fails so that the user can correct them.

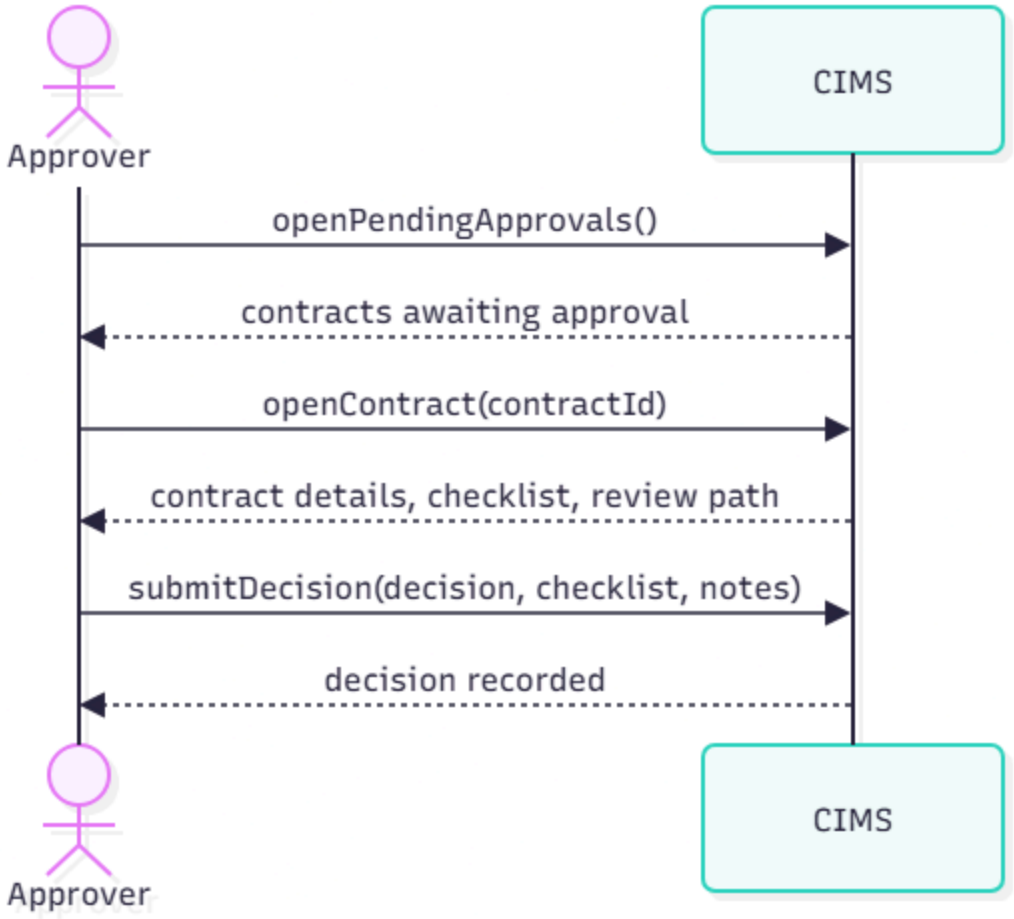


Figure 1.2.3c. System sequence diagram for Review and Approve Contract (main flow).

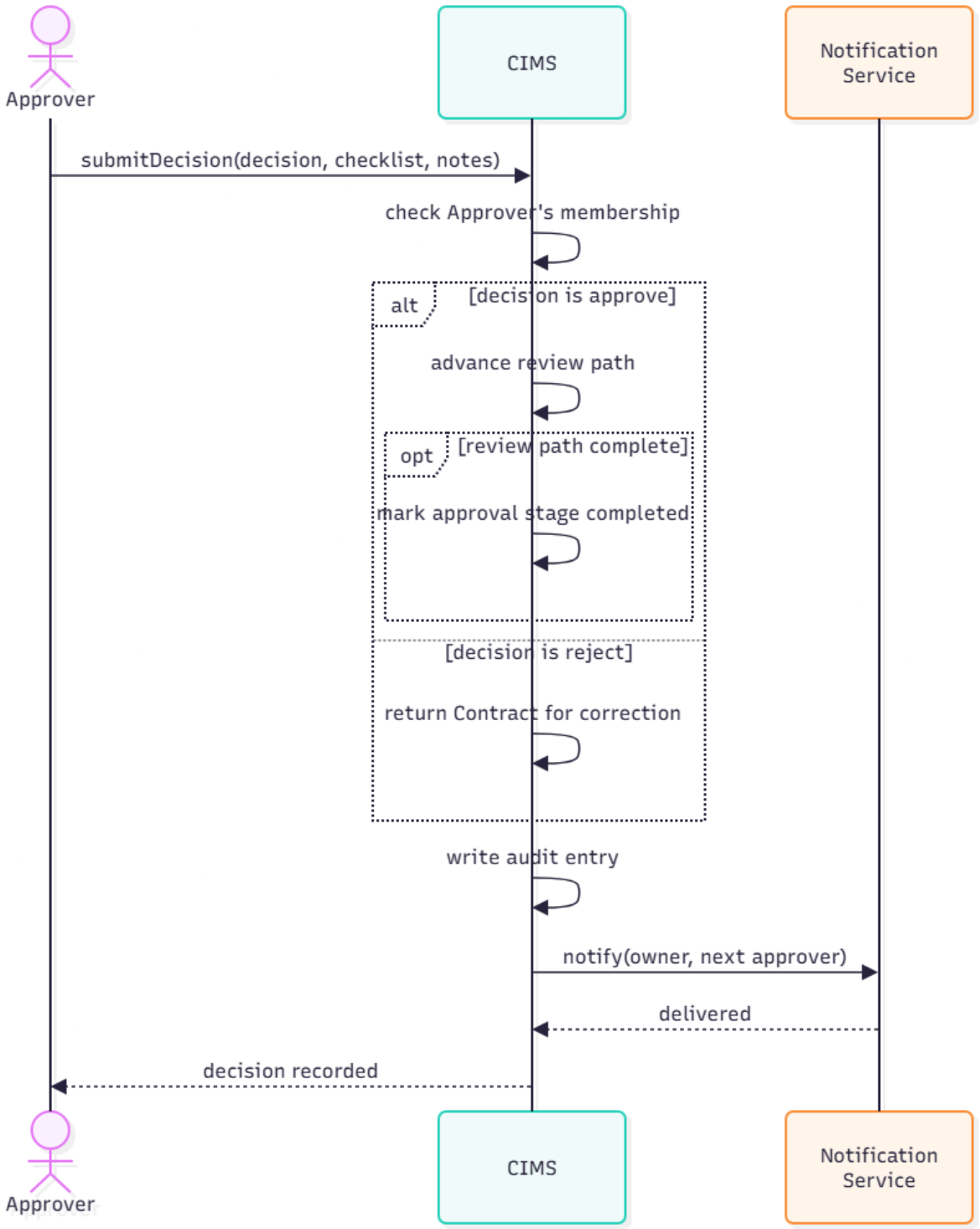


Figure 1.2.3d. Review and Approve Contract: decision handling.

The permission check compares the Contract's Office with the Approver's memberships, so an Approver in one Office cannot decide on another Office's Contract. The status that a rejected Contract returns to is to be decided.

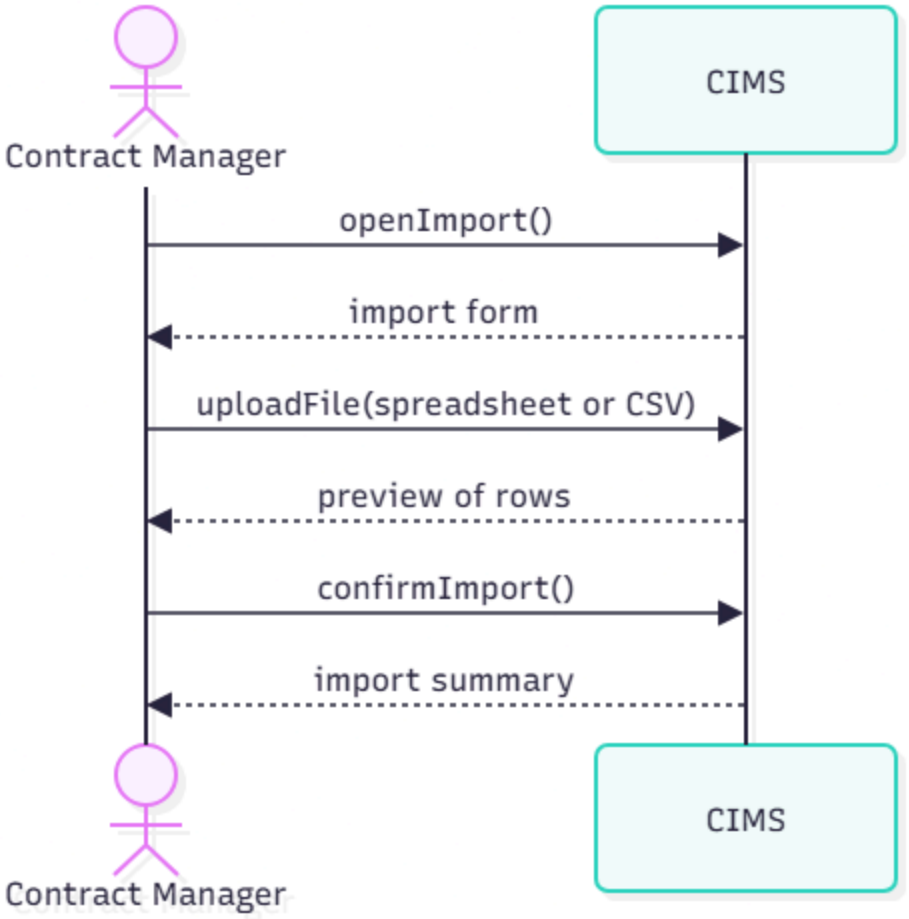


Figure 1.2.3e. System sequence diagram for Import Contracts from Spreadsheet (main flow).

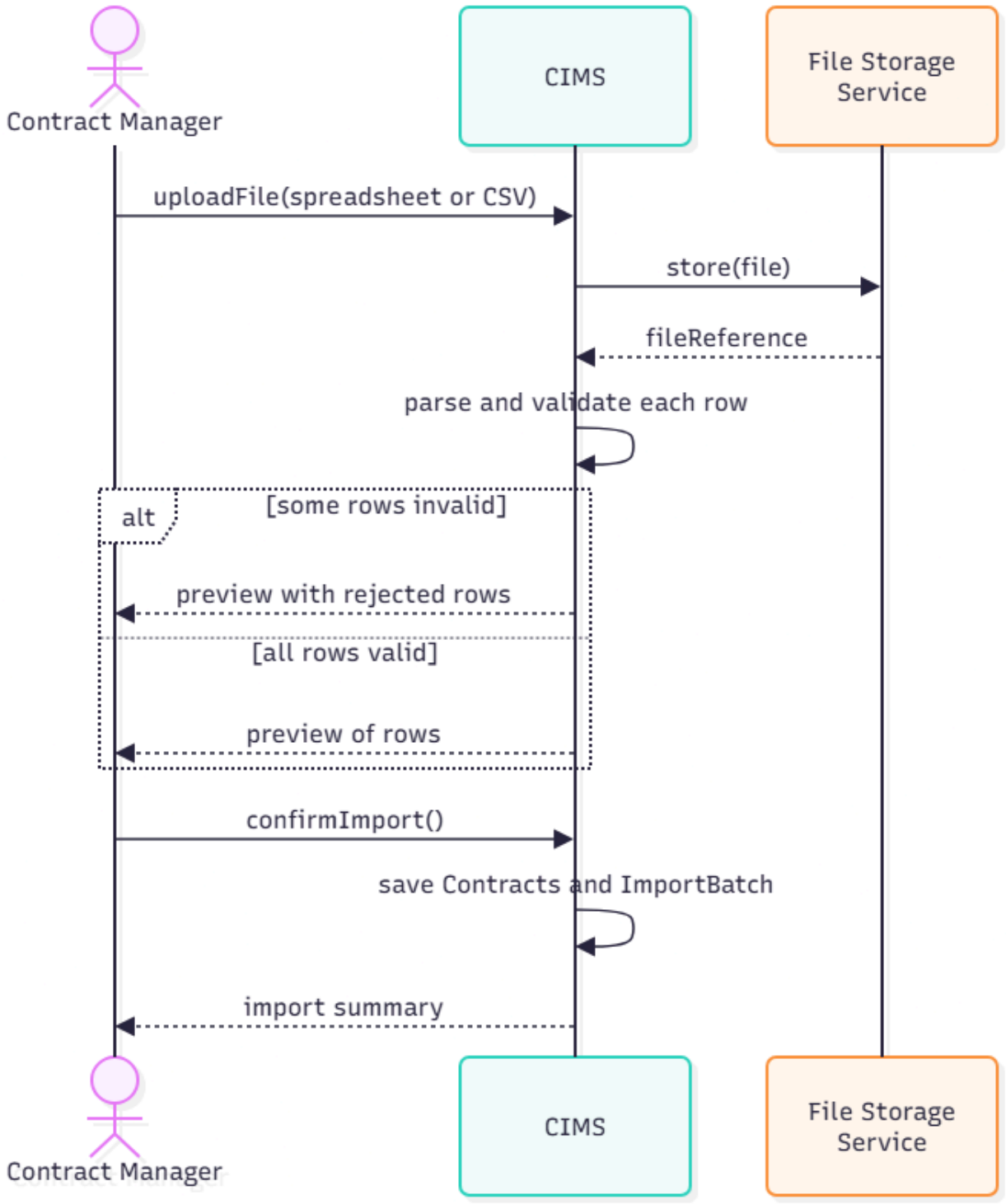


Figure 1.2.3d. Import Contracts from Spreadsheet: validation and saving.

Rows that fail validation are reported with their reasons and are not saved. Only the accepted rows become Contracts after the Contract Manager confirms the import, and the result is recorded as an ImportBatch.

1.3 Configuration Model

The system needs to run on multiple computers. CIMS always runs on at least two kinds of machine: each user's computer, which runs only a web browser, and a server side made up of the application server and the Supabase services (PostgreSQL database, Supabase Auth, and Supabase Storage). For the current client, the application server is the PCED server. Whether the Supabase services run on the same host as the application server or on a separate host is to be decided, and so is where CIMS will be hosted for other offices. Sign-in uses Google Identity through Supabase Auth. No native mobile app is planned; phones and tablets use the same web pages through a browser, so no code runs on a base station or a mobile device other than the browser.

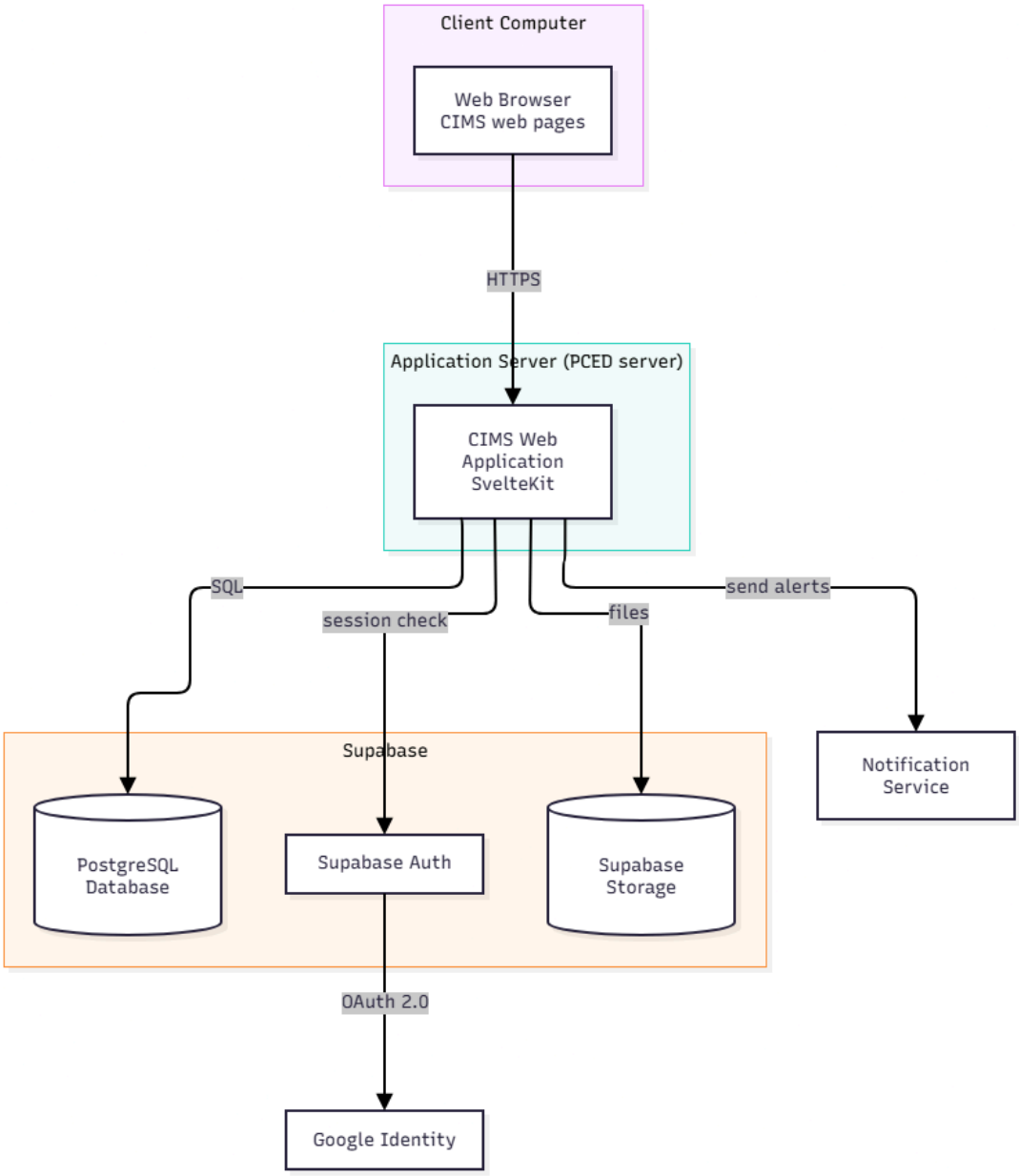


Figure 1.3a. Deployment diagram of CIMS (high level).

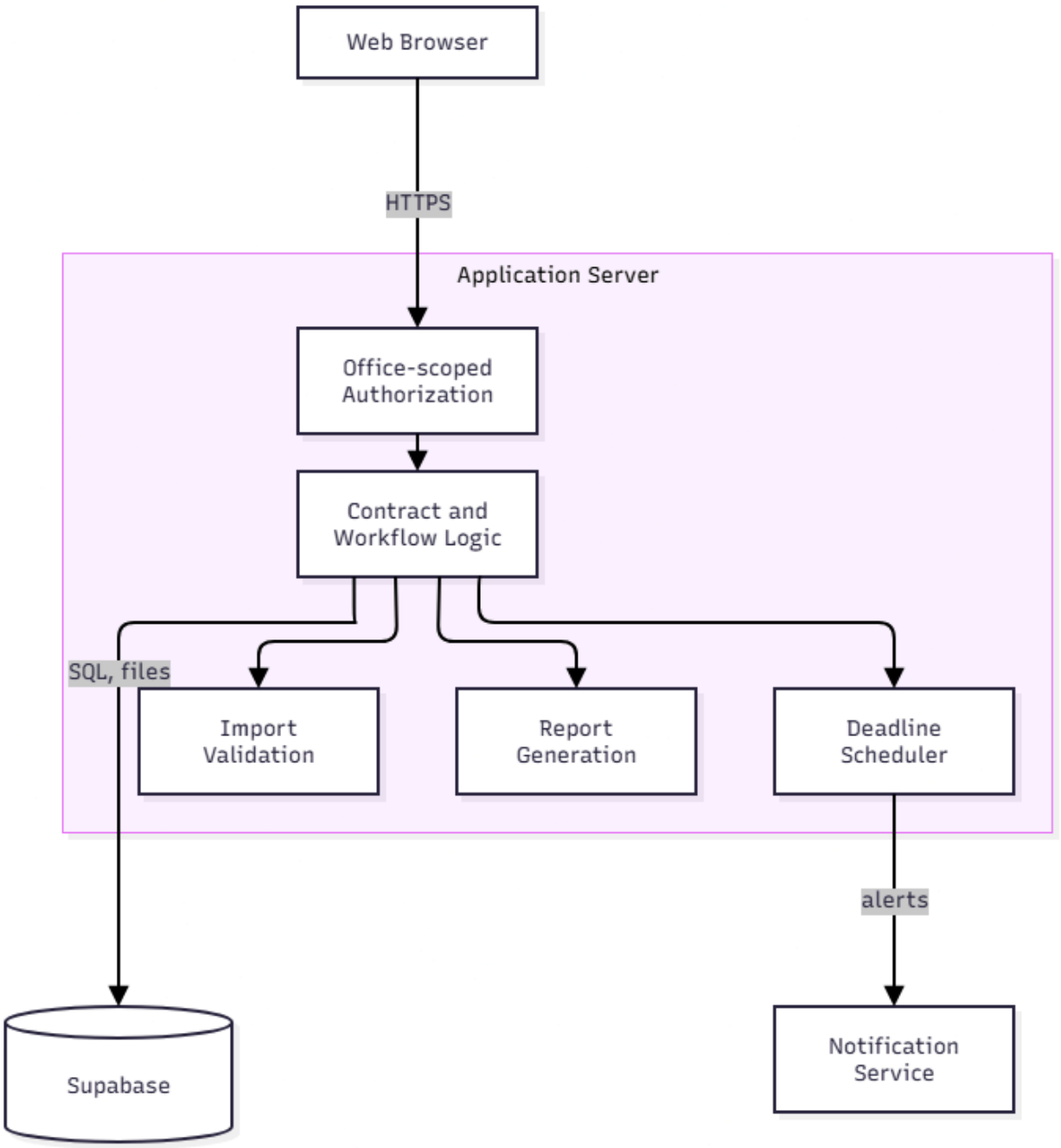


Figure 1.3b. Modules inside the Application Server.

The table below shows which module runs on which machine.

Table 1.3. Modules Running on Which Module.

Module	**Runs on**	**Reason**
Web UI pages, navigation, form display	Client Computer (browser)	Presentation only.
Client-side input checks (required fields, date format)	Client Computer (browser)	Fast feedback only. The server re-checks everything because the client cannot be trusted.
Office-scoped Authorization	Application Server	Must be authoritative and cannot be bypassed from the browser.
Contract and Workflow Logic	Application Server	Business rules and stage transitions.
Import Validation	Application Server	Parses and validates spreadsheet and CSV rows before saving.
Report Generation	Application Server	Builds the Office report from the Office's Contracts.
Deadline Scheduler	Application Server	Runs without a user session.
Database	Supabase	Persistent, shared by all users and Offices.
Authentication Service	Supabase	Credentials are never stored by the web application.
File Storage Service	Supabase	Stores uploaded contract documents.
Message delivery	Notification Service (external)	Sends reminders and alerts.

2 Web App Architecture and Design

2.1 Architecture Design

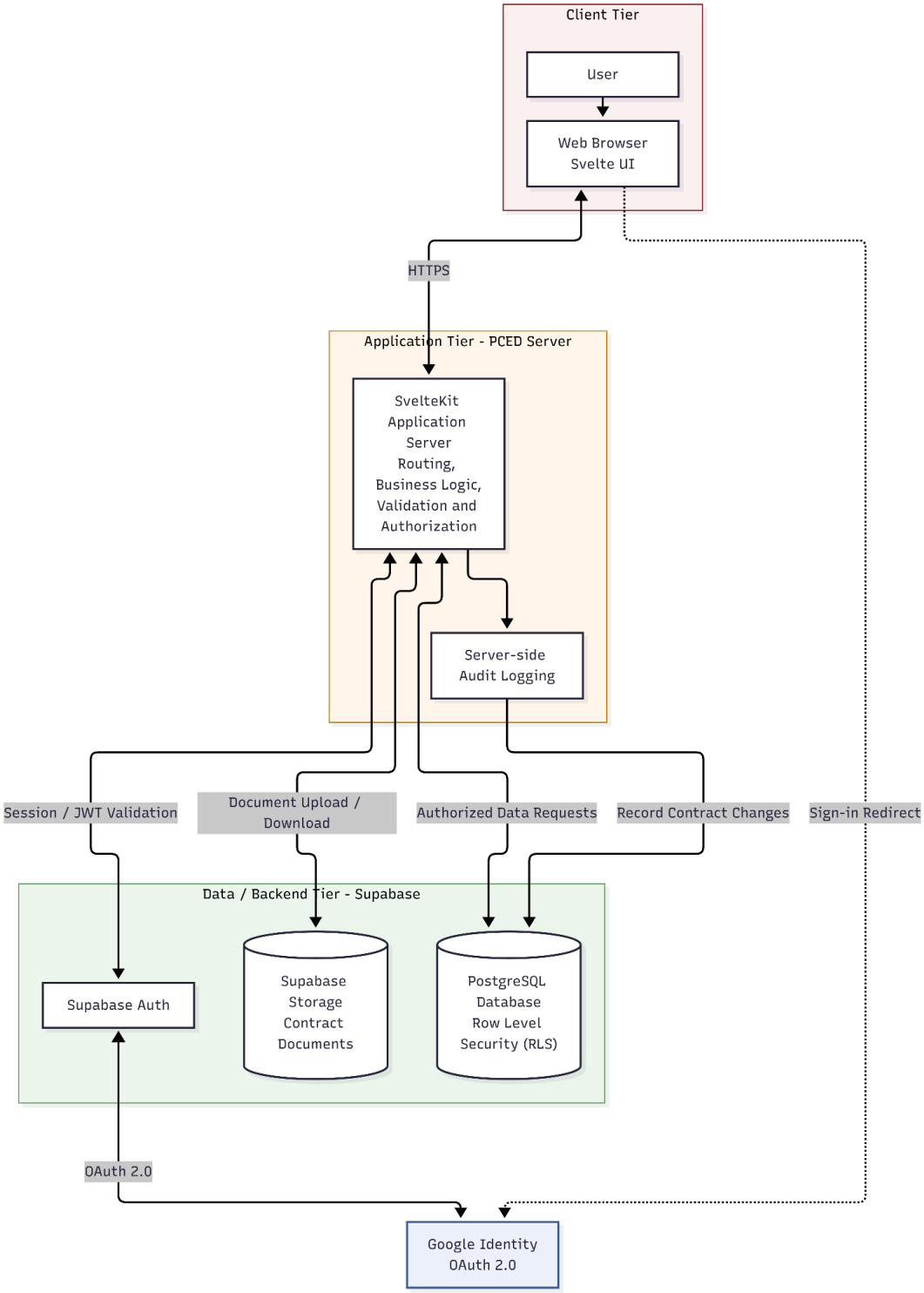


Figure 2.1a. High-Level Diagram of Software Architecture.

The system follows a three-tier client-server architecture consisting of the client, application, and data/backend tiers. Users interact with CIMS through a web browser, where they can manage workflow templates, create and track contracts, upload documents, and search for records. All requests are transmitted securely over HTTPS to the SvelteKit application server which is hosted on the PCED server, in the case of our client.

The application server processes user requests, validates submitted information, and checks permissions before carrying out actions. It also manages contract lifecycle operations and records contract changes in an audit log. When a user signs in, Google Identity authenticates their identity through OAuth 2.0, integrated with Supabase Auth. Supabase provides the PostgreSQL database for application data and Supabase Storage for contract documents. Role-based access is enforced both by the application server and PostgreSQL Row Level Security (RLS), providing an additional layer of protection against unauthorized access.

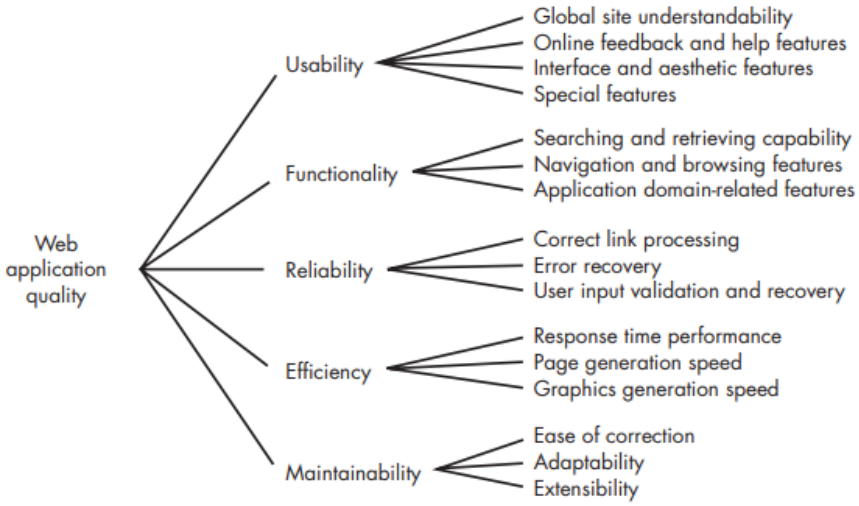


Figure 2.1b. Quality Requirement Tree.

Usability

- *Global site understandability:* CIMS provides a consistent dashboard with persistent navigation for Contracts, Workflows, and Users. All contract types follow the same four-stage lifecycle: Prework, Review & Approval, Signing & Activation, and Postwork.
- *Online feedback and help*: Confirmation prompts for destructive actions, success and error messages, and a user manual linked from the interface help users complete tasks and recover from mistakes.
- *Interface and aesthetic features:* The interface uses consistent components and the UP maroon color palette established by the login screen.
- *Special features:* CSV and spreadsheet imports simplify data entry, while stage tabs show each contract's current lifecycle stage.

Functionality

- *Searching and retrieving:* Users can filter and sort contracts by type, status, and metadata, satisfying functional requirement (FR) 8. Database indexes support efficient retrieval.
- *Navigation and browsing:* Users navigate from the dashboard to the contracts list, contract details, and individual stage tabs, with breadcrumbs to maintain context.
- *Application domain-related:* CIMS supports workflow templates, contract lifecycle tracking, role-based access, and audit logging, covering the system's functional requirements (FR 1-8).

Reliability

- *Correct link processing:* SvelteKit's file-based routing maintains consistent URLs, while a custom 404 page and redirects for unauthorized access support correct navigation.
- *Error recovery:* Failed Supabase requests produce readable error messages. Transactional writes prevent partially saved contracts, while soft deletion and audit logs support mistake recovery and change traceability.
- *User input validation and recovery:* Client-side and server-side validation checks required fields, date ranges, and file types and sizes. Form entries are preserved when errors occur, allowing users to correct mistakes without re-entering their information.

Efficiency

- *Response time performance:* Database indexes, pagination of contract lists, and selective retrieval of necessary columns reduce response times and unnecessary data processing.
- *Page generation speed:* SvelteKit uses server-side rendering for initial page loads and client-side navigation for subsequent interactions.
- *Graphics generation speed:* The interface primarily uses text, tables, and SVG icons with minimal images, keeping graphics processing requirements low.

Maintainability

- *Ease of correction:* Domain-specific modules, centralized error handling, and audit logs help developers identify and resolve problems.
- *Adaptability:* Data-driven workflow templates allow new contract types and stage parameters to be introduced without application code changes. Environment variables manage Supabase configuration, simplifying deployment and project handovers.
- *Extensibility:* New features, such as the planned reporting and dashboard module, can be added through new routes and database tables. Technical documentation is also an explicit project deliverable to support future maintenance and development.

Table 2.1. Architectural Decisions.

Decision	**Rationale**	**Trade-off**
Keep SvelteKit + Supabase	There already exists a prototype; the project plan prioritizes deploying and stabilizing it	Locks the project to Supabase's model, making it a requirement for the team to learn
Role checks in both the server and RLS	Defense in depth for role-based access control	Policies must be kept in sync
Google OAuth only	Fits UP accounts and avoids storing passwords	Depends on Google's availability
Copy the workflow template into each contract when it is created	Editing a template does not break contacts already running on it	More stored data
Optimistic locking (version or updated_at check)	Prevents silent overwrites and data races by concurrent editors	Users may see a "record changed" message
Server-side audit log on every contract change	Satisfies functional requirement 8 and gives traceability	One extra write per action

2.2 Content (Data) Design

2.2.1 Class Diagrams

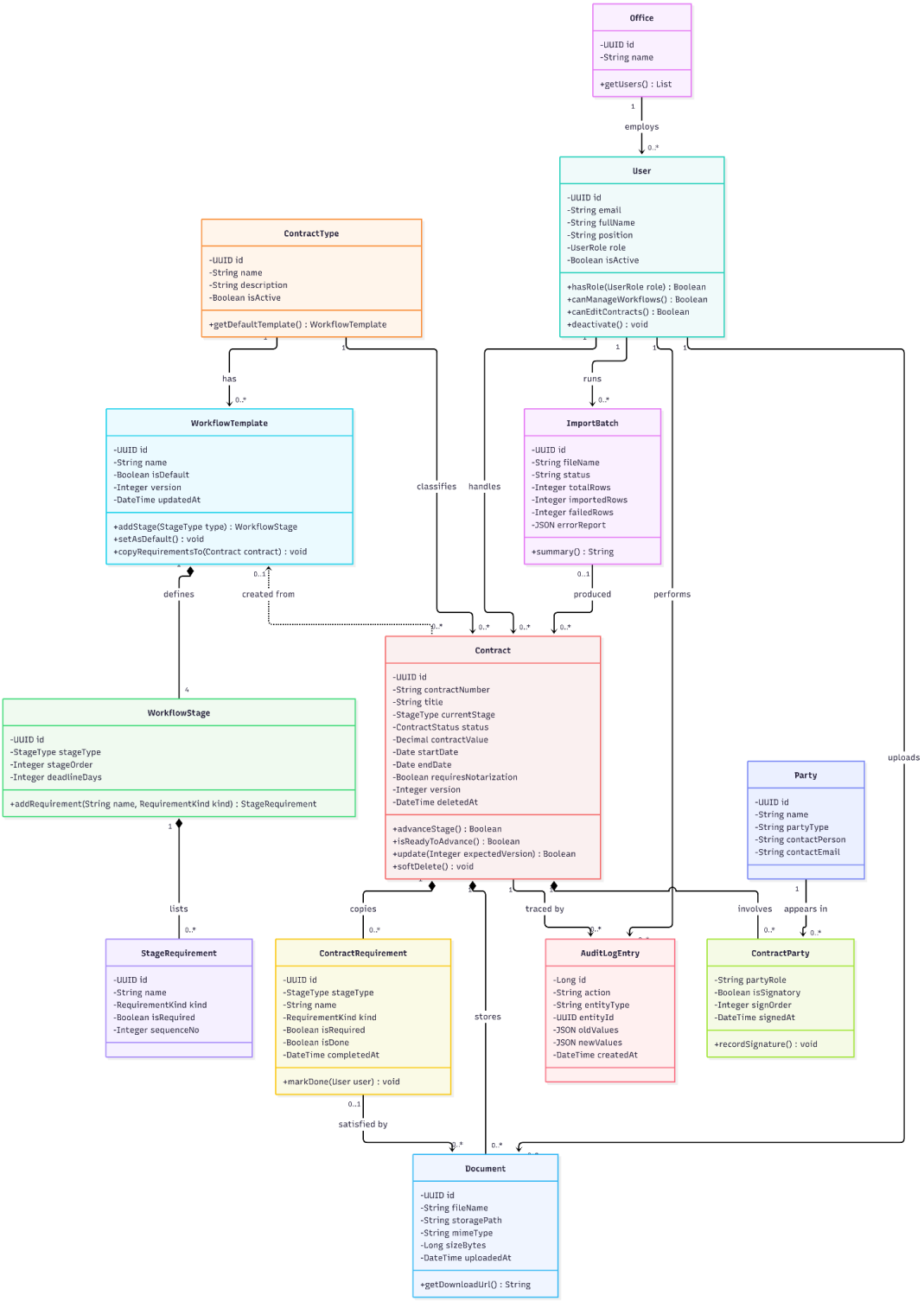


Figure 2.2.1a. Domain Class Diagram.

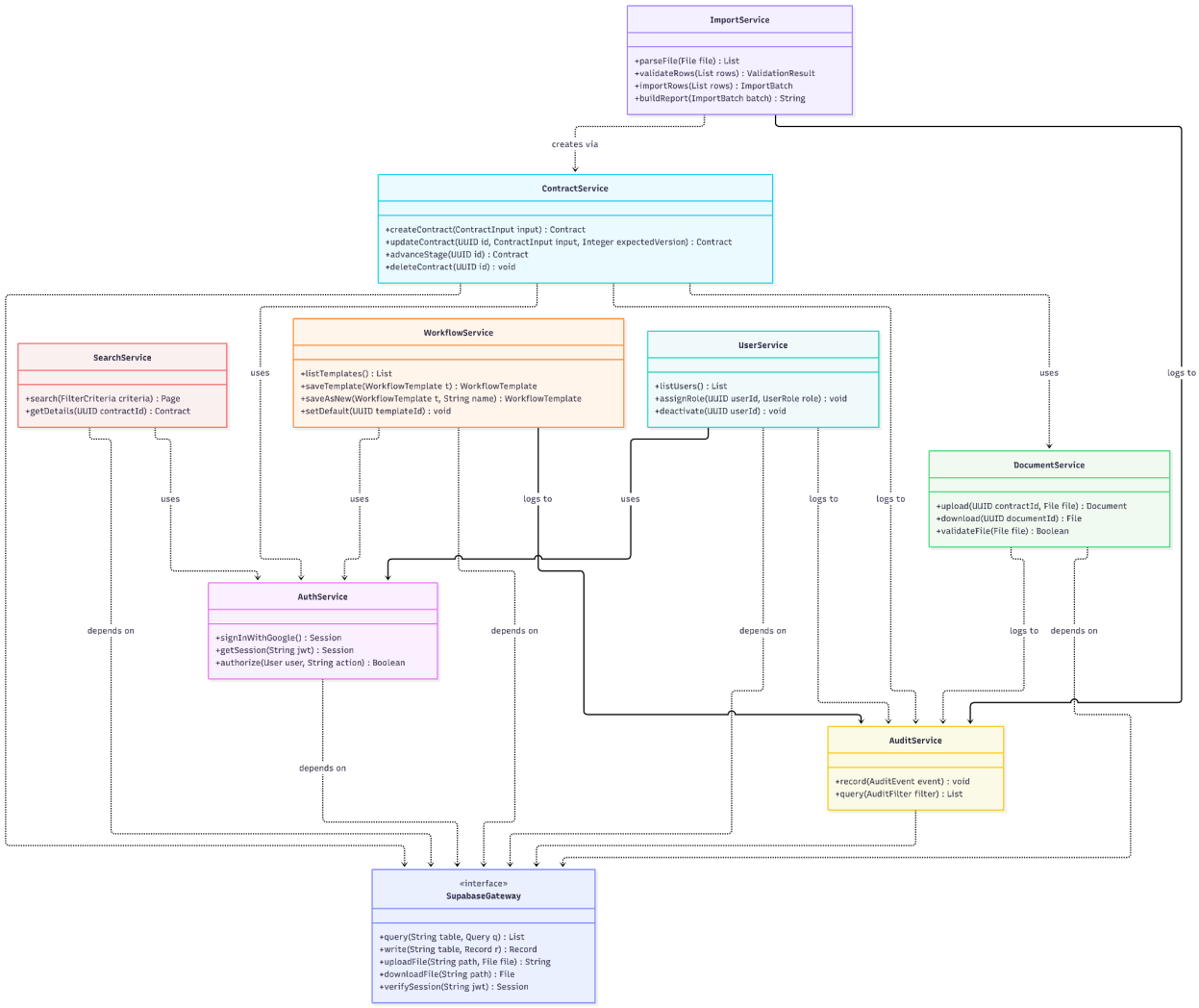


Figure 2.2.1b. Service Class Diagram with Dependencies.

2.2.2 *Entity-Relationship Diagrams*

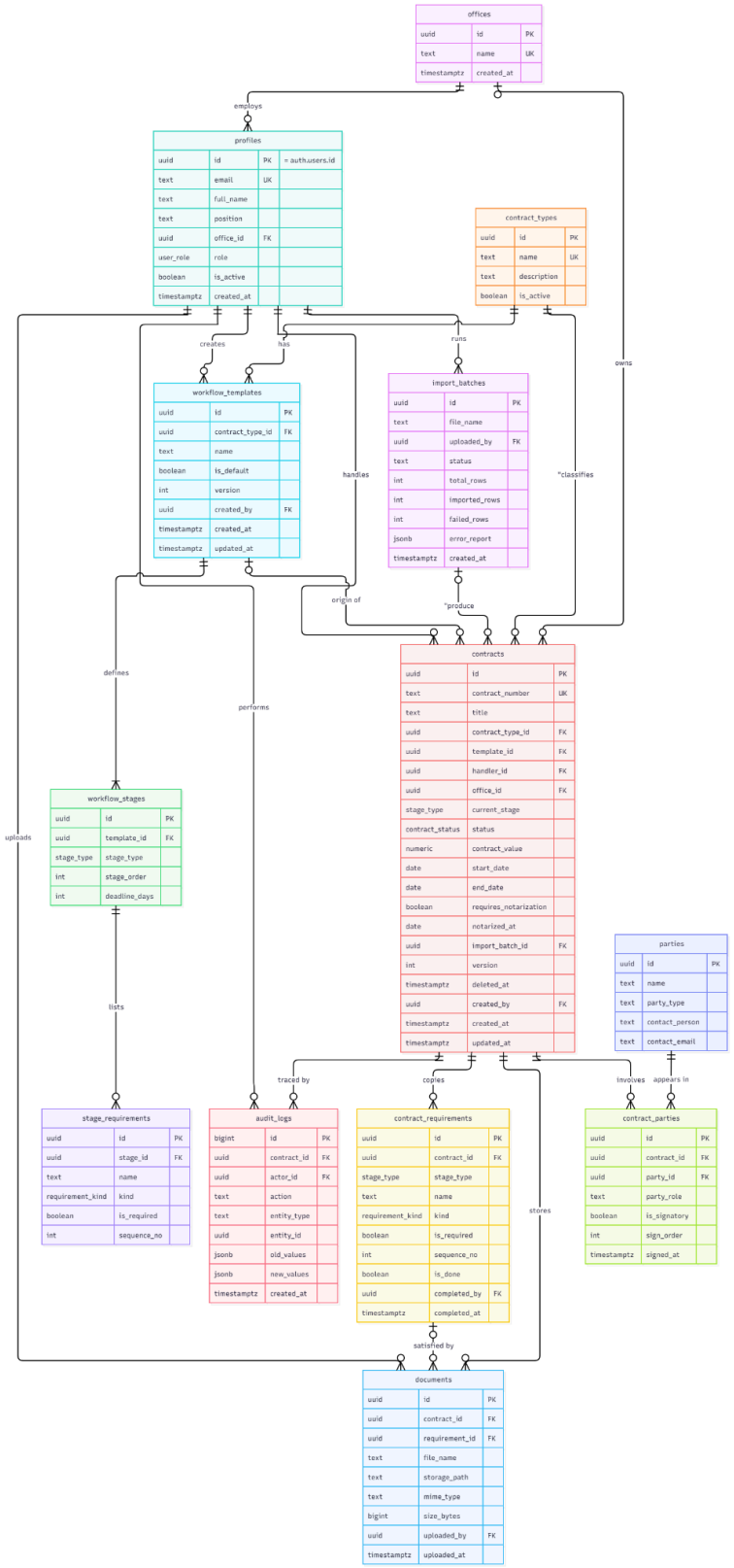


Figure 2.2.2. Entity-relationship Diagram.

2.2.3 *Data Flow Diagrams*

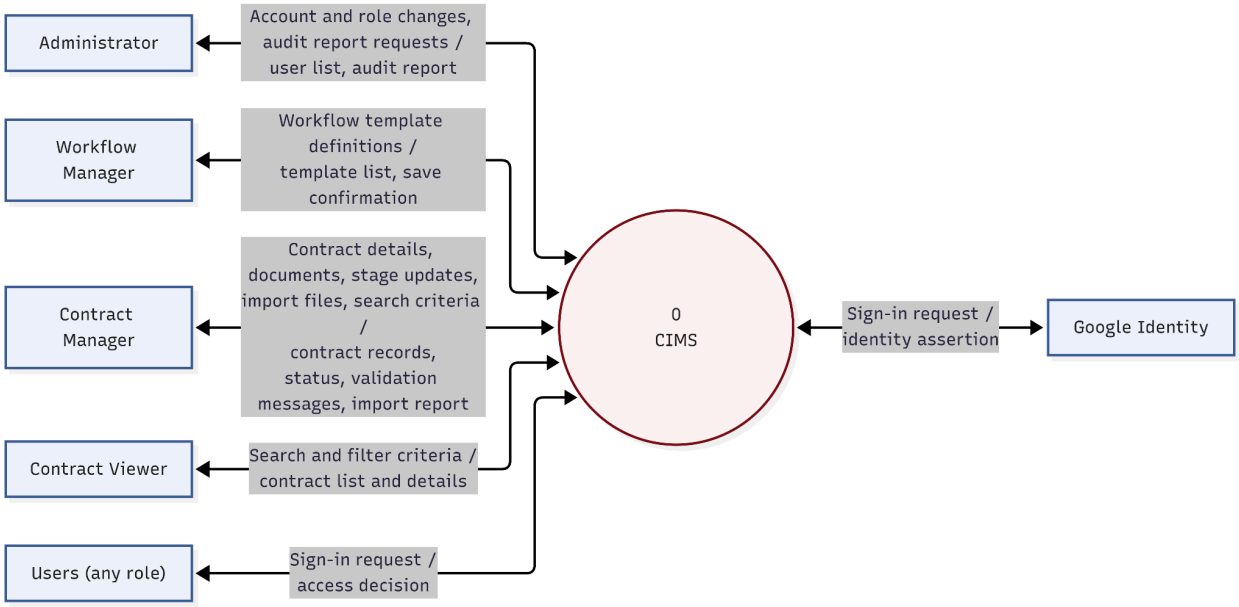


Figure 2.2.3a. Level 0 Data-Flow Diagram.

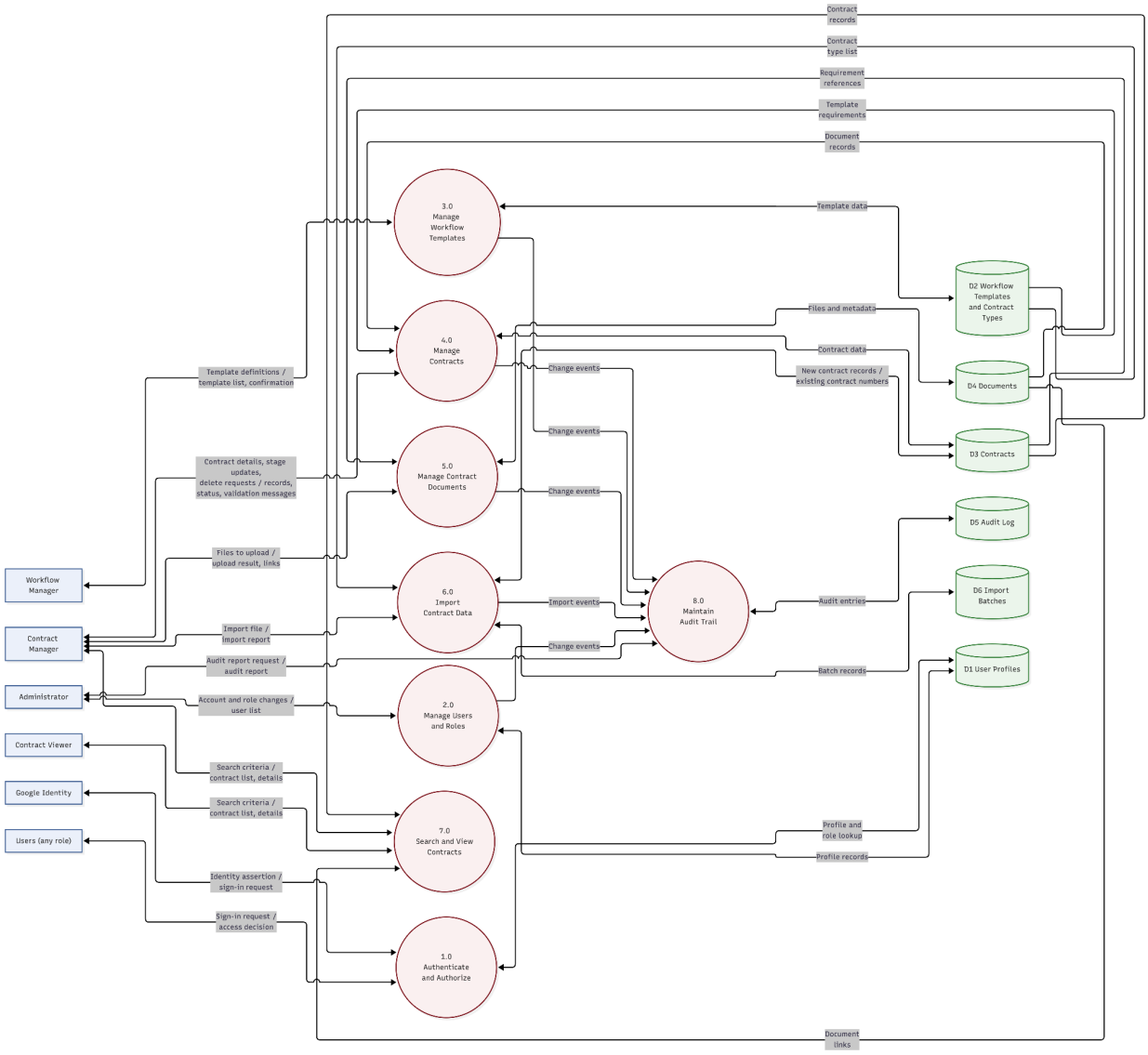


Figure 2.2.3b. Level 1 Data-Flow Diagram.

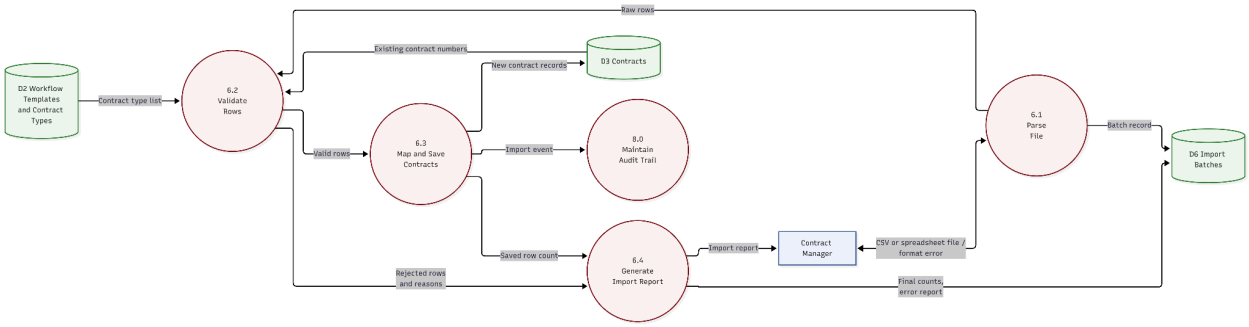


Figure 2.2.3c. Level 2 Data-Flow Diagram (Manage Contracts).

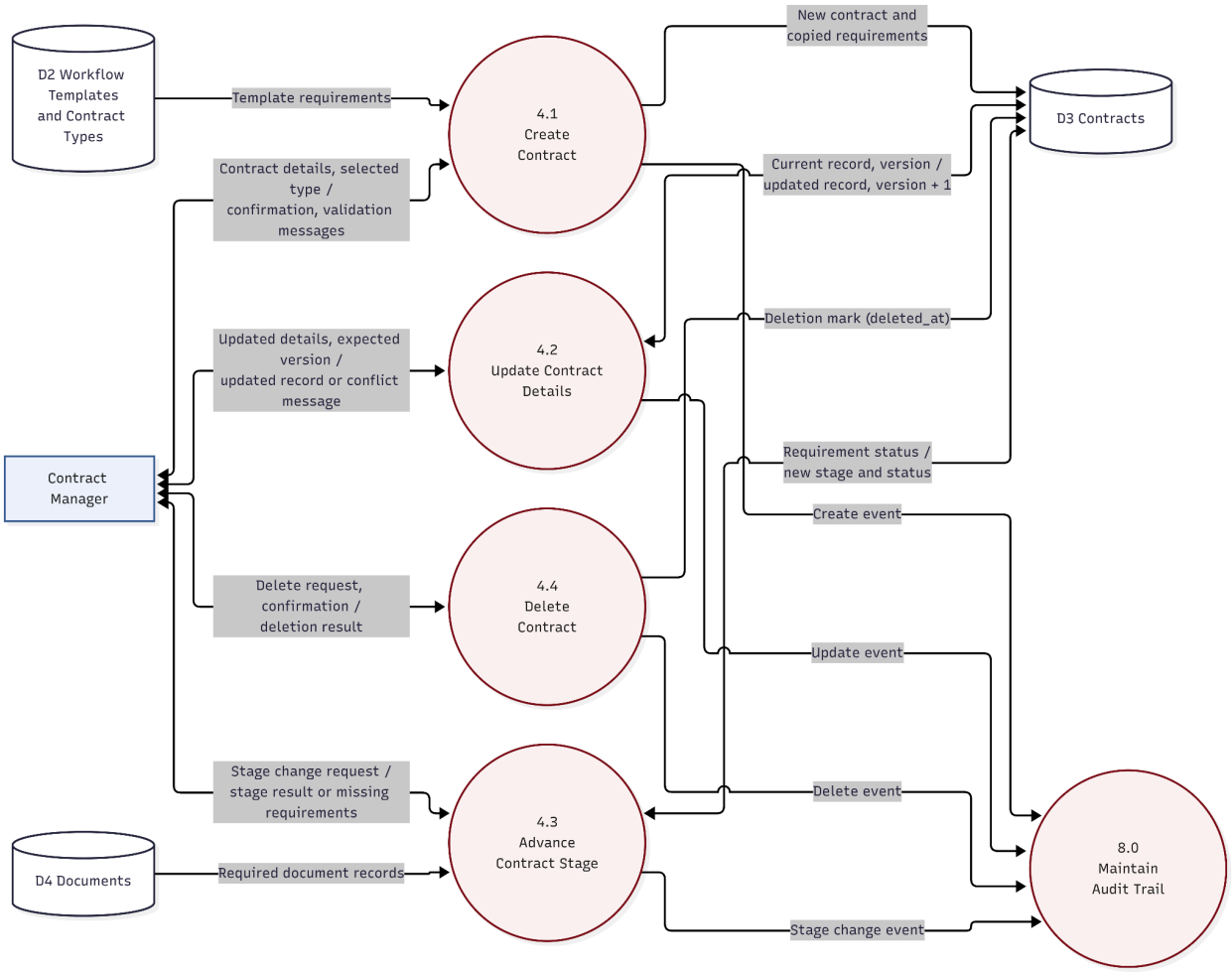


Figure 2.2.3d. Level 2 Data-Flow Diagram (Import Contracts).

2.3 Component-level Design

2.3.1 Algorithms

CIMS uses several processing procedures to support contract management, workflow configuration, and data importation. Two notable algorithms in the current implementation are CSV parsing and contract creation and storage.

2.3.1.a CSV Parsing Algorithm

The system includes a CSV parsing utility that converts CSV text into structured data that can be processed by the application. The parser handles comma-separated values, quoted fields, escaped quotation marks, and line breaks within quoted fields.

The algorithm follows these steps:

1. Receive the CSV text as input.
2. Read the text character by character and separate it into rows while accounting for quotation marks.
3. Process each row by separating individual fields using commas outside quoted fields.
4. Remove surrounding whitespace from parsed field values.
5. Treat the first parsed row as the column headers and convert the headers to lowercase.
6. Exclude empty rows and return the headers and remaining rows as structured arrays.
7. If the input contains no nonempty rows, return empty header and row arrays.

This algorithm allows CIMS to handle CSV data containing commas and line breaks within quoted fields without incorrectly splitting those values. The resulting data can then be used by the application's import functionality.

2.3.1.b Contract Creation and Storage Algorithm

CIMS represents contract information using a structured object containing the contract name, selected workflow ID, and information associated with the four workflow stages: Prework, Review and Approval, Signing and Activation, and Postwork.

The current implementation follows these steps when saving a contract:

1. Receive the contract information from the application.
2. Insert a new record into the contracts table, using the contract title, draft status, and standard type.
3. Retrieve the newly created contract record and its generated identifier.
4. If the initial insertion fails, return the error and stop the remaining storage operations.
5. If the insertion succeeds, store the corresponding Prework, Review and Approval, Signing and Activation, and Postwork information in their respective database tables.
6. Return the newly created contract record to the application.

The implementation uses asynchronous database operations to store the information associated with the four workflow stages concurrently. This allows the system to initiate these operations without waiting for each one to finish before starting the next.

However, the current implementation does not explicitly check the individual errors returned by these four subsequent insert operations. Therefore, successful creation of the main contract record does not necessarily guarantee that all associated workflow information was saved successfully.

2.3.2 Data Structures

CIMS uses arrays, objects, and reactive stores to organize and manage contract information. These data structures allow the system to handle contracts with varying requirements, approval stages, signing parties, and milestones.

Arrays are used to store collections of related elements, such as approval items, signing parties, and postwork milestones. They are suitable for these purposes because the number of elements may vary between contracts and workflow templates. Arrays also allow the system to iterate through and process each element individually.

Objects are used to group related information into a single structure. For example, the system defines a ContractData interface that specifies the contract name, workflow identifier, and information associated with the four workflow stages: Prework, Review and Approval, Signing and Activation, and Postwork. Nested objects and arrays allow each stage to contain its own information and collections of items. This structure makes contract data easier to organize, access, and modify.

The system also uses a Svelte writable store named contractStore to maintain shared contract information. This allows multiple components to access and update the same application state, with changes reflected reactively in components that subscribe to the store. This approach is appropriate for coordinating contract information across different parts of the user interface.

For persistent storage, CIMS uses Supabase, which provides access to a PostgreSQL relational database. Contract and workflow information is organized into tables, including contracts, workflows, contract_preworks, contract_approvals, contract_activations, and contract_postworks. Separating the main contract records from their associated workflow-stage information allows the system to store and retrieve different aspects of a contract independently.

Overall, these data structures support flexibility and maintainability. Arrays and nested objects accommodate workflows with varying numbers of elements, TypeScript interfaces help maintain consistent data structures during development, reactive stores support shared application state, and relational database tables provide persistent storage for contract and workflow information.

2.4 Interface Elements / Usability Design

2.4.1 Interface Design - Menus and links

Upon entering the UPSE Contracts Information Management System (CIMS), users are greeted with a login page through which they can authenticate using their Google account.

After logging in, the user will see a web-based interface with a navigation bar located at the top of the page. The navigation bar includes three main sections, Home, Workflows, and Contracts, plus a Sign Out button.

The active section is highlighted in dark red to help users identify which part of the system they are currently accessing.

The Home page displays the currently logged-in user's account details, including their full name, username, email address, and assigned group. It also contains a Registered Users section that is only accessible by administrator accounts to view registered user accounts.

Users are assigned to groups that determine their access permissions within the system, allowing access to be managed according to their responsibilities.

The Workflows page allows users to create and manage workflow templates for different types of contracts and agreements. Users can select an existing workflow from a dropdown menu, create a new workflow using the Add Workflow button, or delete the currently selected workflow.

Each workflow is divided into four stages: Prework, Review and Approval, Signing and Activation, and Postwork. These stages are presented through a horizontal navigation bar, allowing users to move between the different parts of the workflow configuration.

The Prework stage allows users to configure default contract requirements, including field names and data types. The Review and Approval stage provides controls for configuring approval stages and checklist items. The Signing and Activation stage allows users to specify the parties whose signatures are required, while the Postwork stage provides controls for defining output milestones and contract termination details.

The Contracts page displays a list of contract records in a table. Users can search for specific contracts and filter the records by year, type, and status. The page also provides options to import contract records from CSV files, export contract records, and create new contracts. Selecting a contract opens its details page, where users can review the available information and access the Edit, Delete, and Back buttons.

When creating a new contract, users are first presented with a page where they can search for and select an existing workflow template. After selecting a template, the system displays a summary of its configured requirements, approval stages, signing parties, and postwork milestones. Users can then enter the information required for the new contract, attach relevant files, and save or cancel the contract creation process.

2.4.2 Aesthetic Design - Layout and graphics

The system uses a consistent visual design across its main pages. The interface primarily uses a white background with light-gray content areas, dark-green headings, and dark-red accents.

The University of the Philippines Diliman Crest, followed by a UP School of Economics label is displayed in the upper-left corner of the navigation bar.

Green and red are also used for buttons and highlighted interface elements, while other colors distinguish actions such as importing, exporting, saving, and deleting records. Content is organized into separate sections and cards with rounded corners, subtle shadows, and visible spacing.

Form fields and tables use consistent borders and alignment, helping users distinguish between different types of information.

The system also uses icons alongside navigation items and buttons to help communicate their functions.

On the Contracts page, information is organized in a tabular format with columns for the contract title, creation date, last modification date, type, and status. Search fields, dropdown filters, and colored status indicators allow users to locate and identify contract records more easily.

On the Workflows page, the four workflow stages are displayed as separate horizontal tabs. The selected stage is highlighted in dark red, while the remaining stages use a lighter background. Input fields, checkboxes, dropdown menus, and action buttons are used to configure the requirements and processes associated with each workflow.

Overall, the interface uses consistent colors, spacing, and layout patterns to maintain a recognizable appearance across the system. These design choices help organize contract information and make the system's main functions easier to identify.

2.4.3 Navigation Design

The system organizes its main functions into three sections: Home, Workflows, and Contracts. This structure allows users to move between account information, workflow configuration, and contract management through the navigation bar.

Several typical user tasks can be used to evaluate the system's navigation design:

Table 2.4.3a. User Action Steps and Input Metrics for Viewing an Existing Contract.

Scenario 1: Viewing an Existing Contract		
Step	**Mouse clicks**	**Assumed Keystrokes**
1. Click on Contracts from the navigation bar.	1	0
2. Scroll to find the desired contract	0	5
3. Click on the desired contract	1	0

Table 2.4.3b. Navigation vs. Data Entry Effort Analysis for Viewing an Existing Contract.

Action Category	**Mouse clicks**	**Assumed Keystrokes**	**Total Actions**
UI Navigation	2	5	7
Clerical Data Entry	0	0	0
TOTAL (In Percentage)			
UI Navigation		100%	
Clerical Data Entry		0%	

Table 2.4.3c. User Action Steps and Input Metrics for Viewing an Existing Contract (Search/Filter Method).

Scenario 1 (Alternative): Viewing an Existing Contract		
Step	**Mouse clicks**	**Assumed Keystrokes**
1. Click on Contracts from the navigation bar.	1	0
2. Search for the desired contract or apply the available filters.	1	12
3. Select the contract from the records table.	1	0

Table 2.4.3d. Navigation vs. Data Entry Effort Analysis for Viewing an Existing Contract (Search/Filter Method).

Action Category	**Mouse clicks**	**Assumed Keystrokes**	**Total Actions**
UI Navigation	3	12	0
Clerical Data Entry	0	0	0
TOTAL (In Percentage)			
UI Navigation		100%	
Clerical Data Entry		0%	

Table 2.4.3e. User Action Steps and Input Metrics for Creating a New Contract.

Scenario 2: Creating a New Contract		
Step	**Mouse clicks**	**Assumed Keystrokes**
1. Click on Contracts from the navigation bar.	1	0
2. Click on the “Create Contract " button	1	0
3. Search for a suitable workflow template	1	10
4. Select the Desired Template	1	0
5. Navigate through the workflow stages and enter the required information	15	50
6. Save the contract	1	0

Table 2.4.3f. Navigation vs. Data Entry Effort Analysis for Creating a New Contract.

Action Category	**Mouse clicks**	**Assumed Keystrokes**	**Total Actions**
UI Navigation	5	10	15
Clerical Data Entry	15	50	65
TOTAL (In Percentage)			
UI Navigation		18.75%	
Clerical Data Entry		81.25%	

Table 2.4.3g. Action Steps and Input Metrics for Creating a New Workflow Template.

Scenario 3: Creating a New Workflow Template		
Step	**Mouse clicks**	**Assumed Keystrokes**
1. Click on Workflows from the navigation bar	1	0
2. Click on the “Add Workflow” button	1	0
3. Type the new workflow name	1	10
4. Configure Prework requirements	10	10
5. Switch to Review and Approval	2	0
6. Configure checklist items under Review and Approval	10	30
7. Switch to Signing and Activation	2	0
8. Configure required signing parties under Signing and Activation	10	30
9. Switch to Postwork	2	0
10. Configure milestones and termination details under Postwork	15	100
11. Save the workflow	2	0

Table 2.4.3h. Navigation vs. Data Entry Effort Analysis for Creating a New Workflow Template.

Action Category	Mouse clicks	Assumed Keystrokes	Total Actions
UI Navigation	10	0	3
Clerical Data Entry	46	180	226
TOTAL (In Percentage)			
UI Navigation		4.24%	
Clerical Data Entry		95.76%	

Table 2.4.3i. User Action Steps and Input Metrics for Editing an Existing Workflow Template.

Scenario 4:Editing an Existing Workflow Template		
Step	**Mouse clicks**	**Assumed Keystrokes**
1. Click on Workflows from the navigation bar	1	0
2. Select the desired workflow to be edited	2	0
3. Edit Prework requirements	10	10
4. Switch to Review and Approval	2	0
5. Edit checklist items under Review and Approval	10	30
6. Switch to Signing and Activation	2	0
7. Edit required signing parties under Signing and Activation	10	30
8. Switch to Postwork	2	0
9. Edit milestones and	15	100

termination details under Postwork		
10. Save the workflow	2	0

Table 2.4.3j. Navigation vs. Data Entry Effort Analysis for Editing an Existing Workflow Template.

Action Category	Mouse clicks	Assumed Keystrokes	Total Actions
UI Navigation	11	0	4
Clerical Data Entry	45	170	215
TOTAL (In Percentage)			
UI Navigation		4.87%	
Clerical Data Entry		95.13%	

The system uses a combination of navigation links, buttons, dropdown menus, tabs, search fields, and form inputs to support these tasks. The horizontal tabs on the Workflows page allow users to move between configuration stages without navigating to separate main pages. Similarly, the Contracts page combines search and filtering controls with a table of records, reducing the need to navigate through multiple pages when locating a contract.

Note that the keystroke counts presented in the tables for the navigation and data entry effort analysis are estimates based on representative inputs for each scenario. Actual keystroke counts may vary depending on the length and content of the information entered by the user. The reported percentages are therefore estimates of the interaction effort required to complete each task.